Ders 2 Genişletilmiş Notu: Hata Sezme ve Düzeltme Kodları ve Sezgisel Sıkıştırma

Ders 2 Genişletilmiş Notu: Hata Sezme ve Düzeltme Kodları ve Sezgisel Sıkıştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Önceki Dersin Tekrarı
- 2. Hata Bulma ve Düzeltmeye Giriş
- 3. Hata Türleri ve Hata Denetim Yöntemleri
- 4. Tekrarlama ile Hata Sezme ve 2 Bit → 5 Bit Kodu
- 5. Hamming Kodlama
- 6. Eşlik Sınaması (Parity)
- 7. CRC (Cyclic Redundancy Check)
- 8. MD5 ve SHA
- 9. Günlük Hayattan Kontrol Rakamlı Kodlar: UPC ve POSTNET
- 10. Sezgisel Sıkıştırma: Braille, Mors, Kısaltmalar
- 11. Sparse String Sıkıştırma
- Örnek Sorular

1. Önceki Dersin Tekrarı

Hoca derse, geçen haftadan akılda kalan anahtar kavramları öğrencilere sorarak başladı. Amaç, iyi oturmamış noktaları yeniden vurgulamaktı.

1.1 Veri sıkıştırma nedir?

Kısaca, yerden ve zamandan kazanmak için yapılan işleme veri sıkıştırma denir. “Yer” fiziksel saklama alanıdır; “zaman” verinin bir yerden bir yere iletilme süresidir. Hoca, sıkıştırmanın çoğu zaman tek bir yöntemle yapılmadığını ekledi: yöntemin önünde ya da arkasında ön işlemler vardır. Örneğin görüntü sıkıştırmada önce mutlaka bir nicemleme (quantization) işlemi yapılır; doküman sıkıştırmada önce verinin istatistiği çıkarılır. Bunlar ilerleyen derslerde görülecek.

1.2 Sabit ve değişken uzunluklu kod

Sabit uzunluklu kodlamada bütün sembollere aynı bit uzunluğunda kod verilir. Değişken uzunluklu kodlamada sembollere kullanım sıklıklarına göre farklı uzunlukta kodlar verilir. Hoca bu kavramların bilgi teorisi (information theory) bölümünde daha ayrıntılı işleneceğini söyledi.

1.3 Adaptif ve non-adaptif yöntemler

Adaptif “dinamik”, non-adaptif “dinamik değil” demektir. Non-adaptif yöntem yalnız belli tipte veriyi sıkıştırır; her veriye uymaz. Adaptif yöntem her tür veriye uygulanabilir; parametreleri ve değişkenleri işlem sırasında değişebilir.

1.4 Kayıplı ve kayıpsız yöntemler

Kayıplı (lossy) yöntemde kodlanan veri yeniden çözüldüğünde orijinal hâline dönmez, bazı bilgiler kaybolur. Bu yöntemler özellikle görüntü ve ses sıkıştırmada kullanılır, çünkü gözün ve kulağın görebildiği ve duyabildiği sınırlar vardır; bu sınırların üstündeki bilgi zaten algılanmadığı için atılabilir. Kayıpsız (lossless) yöntemler genelde metinlere uygulanır: metinde tek bir karakterin kaybı bile anlamı değiştirebilir.

1.5 Simetrik ve asimetrik yöntemler

Simetrik algoritmada kodlama algoritması açarken tam tersten çalıştırılır. Asimetrik algoritmada sıkıştırma ve açma aynı algoritma değildir; hızları da farklı olabilir. Bazı algoritmalar hızlı sıkıştırıp yavaş açar, bazıları tersini yapar.

1.6 Akış ve blok, derlem, başarı ölçütleri

Girdi kaynaktan bayt bayt ya da kelime kelime alınabileceği gibi blok blok da alınabilir. Blok blok çalışan yöntemlerde her bloğa farklı bir algoritma uygulanabilir; açarken hangi bloğun hangi algoritmayla kodlandığının bilinmesi için bu bilgi başlık (header) kısmına yazılır.

Derlem (corpus), yeni bir sıkıştırma algoritmasının denenmesi gereken ortak veri setidir. Görüntü işleme ve ses üzerinde çalışanların da kendi veri setleri vardır; hoca bunlara "baseline" dendiğini söyledi. Herkes aynı derlemi kullanınca kimin ilerde kimin geride olduğu görülür; derlem çok farklı verilerden oluştuğu için algoritma farklı girdilerde de sınanmış olur.

Başarı birimi metinde bpc (bit per character, bir karakterin ortalama kaç bitle kodlandığı), görüntüde bpp (bit per pixel) idi. Metinde sıkıştırma oranı çıktının boyunun girdinin boyuna oranıdır. 100 birimlik veri 30 birime inerse:

$$\text{oran} = \frac{30}{100} = 0{,}3, \qquad \text{başarı} = (1 - 0{,}3) \cdot 100 = \%70$$

Görüntüde ise başarı çıktı/girdi oranıyla değil, açılan görüntünün bozulmasına bakılarak ölçülür. Sıkıştırılan görüntünün ($\hat{I}$) boyutları değişmez; içindeki renk sayısı azalır. Açılan görüntü orijinale benziyorsa sıkıştırma iyidir. Bunun için orijinal ve açılmış piksel değerlerinin farklarının karesi alınır, görüntü boyutuna oranlanıp karekökü alınır (root mean square error). Hoca karenin neden alındığını şöyle açıkladı: $5 - 2 = 3$, $2 - 5 = -3$; işaretler farklı olsa da değişim 3 birimdir, kare alınınca yön farkı ortadan kalkar. Sayısal analizdeki en küçük kareler yöntemi de aynı fikre dayanır. Bir de sinyal-gürültü oranı (SNR) vardır; ikisi de kullanılır ama sonuçları ters yöndedir: biri küçükken diğeri büyük çıkar.

2. Hata Bulma ve Düzeltmeye Giriş

Hoca sıkıştırma yöntemlerine kısa bir ara verip hata bulma ve düzeltme konusuna geçti. Bu aslında sinyal işleme ve haberleşme (komünikasyon) derslerinde çok daha derin işlenen bir konudur; bu derste yalnız sıkıştırmayı ilgilendiren önemli kavramlar ele alınır.

2.1 Sıkıştırma ile hata sezme arasındaki ikilem

Veri, boyutunu küçültmek için sıkıştırılır. Hata sezme ise veri bir noktadan diğerine gönderilirken oluşabilecek bozulmaları anlamak için veriye ek bitler eklemektir. Yani bir yandan veri küçültülmeye çalışılırken öbür yandan büyütülür. Hoca bunu bir ikilem (dilemma) olarak tanımladı: veriyi küçültmek mi istiyoruz, yoksa karşı tarafa güvenli gidip gitmediğini mi öğrenmek istiyoruz?

Veri yalnız kendi diskimizde sıkıştırılıyorsa iletim yoktur, sorun da yoktur. Veri iletim hatları üzerinden gönderiliyorsa, örneğin A noktasından B noktasına faks çekiliyorsa, yolda bazı bitler bozulabilir.

2.2 Hata nedir?

Veri iletilirken bazı bitlerde oluşan bozulmalara hata denir (ekrandan, 00:24:30). Ortama bağlı olarak hata olasılığı değişir; bazen verinin küçük bir bölümü bozulur, bazen hiç bozulmaz, bazen tamamı bozulur. Atmosferik olaylar (yıldırım, bir şeyin o an çakması, kum fırtınası gibi) telli ve telsiz ortamlarda hataya yol açar.

Bozulmaların çoğu uygulamalar tarafından kabul edilmez, ama bazıları kabul edilebilir. Hocanın örneği: faksla ortasında 3-4 satır yazı olan beyaz bir sayfa gönderiliyor. Yazıların dışındaki beyaz zeminde bir bölüm kararmış çıkarsa önemli değildir. Bilgi taşıyan kısımda bozulma varsa bu kabul edilemez.

2.3 Tek bitin önemi

Veri içindeki bir bitlik bozulma bütün verinin yanlış anlaşılmasına yol açabilir. Sıkıştırılmış veri, bakınca hiçbir şey anlaşılmayan bir 1-0 dizisidir. Bu dizide bir bit 1 iken 0, ya da 0 iken 1 olursa birçok kod hatalı açılır; açıldığında karşımıza bir metin gelir ama anlamsızdır. Bu yüzden önce bir hata olup olmadığını sezmek gerekir; bunu veriye eklenen ek bitler sağlar.

Hatayı sezmek, onu düzeltebilmek anlamına gelmez. Hata bulunur ama düzeltilemeyebilir. Hatayı bulmanın maliyeti çok düşük, düzeltmenin maliyeti çok yüksektir; düzeltmek yerine "veriyi bana yeniden gönder" denebilir. Bir bitlik hatayı düzeltme olasılığı daha yüksektir, ama bazen bir bitlik hata bile düzeltilemez.

> **Hocanın vurgusu:** Hatayı sezme işlemi hatayı düzeltmeye göre daha kolaydır. Bir bitlik bozulmaları sezmek ve düzeltmek kolaydır; bir bitin üzerindeki bozulmalar sezilse bile düzeltilemez, verinin yeniden gönderilmesi gerekir (ekrandan, 00:24:30).

3. Hata Türleri ve Hata Denetim Yöntemleri

3.1 İki çeşit hata

Slayttaki tanımlar (ekrandan, 00:30:00):

- Patlama hatası (burst error): çevre koşulları nedeniyle alıcıya gerçek olmayan, anlamsız bilgiler gelir (yıldırım vb., 1-100 ms). Hoca bunun milisaniyeler içinde olan bir olayın birçok bitin bozulmasına yol açması olduğunu anlattı.
- Rastgele hata (random error): iletim yolundaki elektriksel gürültü nedeniyle bilgi bloğu içinde rastgele bir bitin bozulmasıdır. Hoca CD'lerin bazen okunamamasını bu tür bozulmaya örnek verdi.

3.2 İletim hatalarının veriye etkisi

- Çok kısa süren girişime spike denir; tek bitlik hataya sebep olur.
- Uzun süren girişim ya da bozulma burst (birden fazla bitlik) hatalara yol açar.
- Sinyal açıkça 1 ya da 0 değilse, yani belirsiz bir alana düşerse buna erasure (silinmiş yer) denir. Hoca burada bir bloğun komple silinmesinden söz ederken binlerce bitin değil, 2-3 bitin kastedildiğini belirtti.

3.3 En çok kullanılan hata denetim metotları

Hoca bütün yöntemleri değil, en popüler olanları göstereceğini söyledi. Slayttaki iki grup (ekrandan, 00:31:40):

1. FEC (Error Correction Code, Forward Error Correction, ileri yönlü hata denetimi)

- Hata denetimi için gönderilen veri kümesine ek bitler ekler. Orijinal veride bozulma olup olmadığı bu bitler sayesinde anlaşılır. Hoca, ek bitlerin kendisi bozulursa "iş işten geçmiş" olacağını da ekledi.
- Hatayı bulur ve gerekirse alıcıda düzeltmeye çalışır; düzeltemezse veriyi yeniden ister.
- Hız kaybı yaşanmaz.
- Çok gürültülü ortamlarda kullanılamaz.

- Yeniden iletimin zor ya da imkânsız olduğu bağlantılarda ve veri kümesinin küçük olduğu uygulamalarda kullanılır. Büyük veri gönderiliyorsa FEC seçilmemelidir.
- Örnekler: katlamalı kodlar, BCH kodları, Hamming kodları, Reed-Solomon kodları. Hoca bunlardan en popüler olan Hamming kodunu göstereceğini söyledi.

2. ARQ (Automatic Repeat Request, otomatik tekrar isteği)

- Hatayı bulma ve bozulan verinin yeniden iletilmesi için alıcı taraftan istekte bulunma işlemidir. FEC'te alıcı düzeltebiliyorsa kendisi düzeltir; ARQ'da bozulma varsa karşı taraftan veriyi ister.
- Uygulamalarda bu teknik kullanılır, çünkü aynı hatayı tespit etmek için gereken bit sayısı, düzeltmek için kullanılan bit sayısından çok daha azdır. Düzeltmeye uğraşırken harcanacak bit, verinin yeniden istenmesinde gelecek bitten fazladır.
- Örnekler: CRC kodları, seri eşlik (parity), blok eşlik, modül toplamı.

Hoca bu derste Hamming kodlarını, CRC'yi ve tek/çift eşliği anlatacağını söyledi.

4. Tekrarlama ile Hata Sezme ve 2 Bit → 5 Bit Kodu

4.1 Ek bit yoksa

Slayttaki örnekte (ekrandan, 00:40:00) gönderilen veri `10110100110`, alınan veri `10100100110`'dır: dördüncü bit 1 iken 0 olarak gitmiştir. Alıcı bu bozulmayı anlayamaz; buranın orijinalinin 1 olduğunu bilemez, 0 olarak alır ve açar. Bitler sembol kodlarıysa araya başka bir sembolün kodu girer. Hoca, bir kelimede "u"nun kodundaki bir bitin değişmesiyle yerine "a"nın kodunun çözüldüğünü düşünmemizi istedi: okuyan kişi bunu belki tahmin eder ama alıcı kesinlikle anlayamaz. Tek bitlik bozulma sezilemedi bile.

4.2 Her biti iki kez yazmak (1 bit → 2 bit)

Her bit iki kez gönderilir: 1 yerine `11`, 0 yerine `00`. Gönderilen `11 00 11 11 00 11 00 00 11 11 00`, alınan `11 00 11 01 00 11 00 00 11 11 00` olsun. Alıcı `01` çiftini görünce hata olduğunu anlar, çünkü her bit kopyasıyla gönderilmiştir. Ama düzeltemez: bu çift `00` iken mi `01` oldu, yoksa `11` iken mi? Bilinemez. Sonuç: hata sezilir ama düzeltilemez.

4.3 Her biti iki kopyayla yazmak (1 bit → 3 bit)

Her bit üç kez gönderilir: `111 000 111 111 000 111 000 000 111 111 000`. Alınan veride dördüncü grup `011` gelmiştir. Alıcı hatayı sezer, çünkü üç bitin aynı olması gerekir. Tek bit bozulduğu için çoğunluğa bakarak düzeltebilir: ters olan 0'dır, orijinal `111`'dir. Birden fazla bit bozulursa hangisinin bozulduğu bilinemez, düzeltilemez. Sonuç: hata sezilir, bir bitlik bozulma varsa düzeltilir, aksi durumda düzeltilemez.

Hoca burada dersin başındaki ikilemi hatırlattı: bozulmayı anlamak için 1 biti 2 bite, düzeltebilmek için 3 bite çıkardık. "Nerede kaldı benim sıkıştırma?" Hep bu ikilemle karşılaşılır: sıkıştırma mı, güvenlik mi? Veri dinamik olarak bir noktadan diğerine aktarılıyorsa güvenliğe mutlaka dikkat edilmelidir; yalnız fiziksel ortamda sıkıştırılıyorsa bunlarla uğraşmaya gerek yoktur.

4.4 Her 2 bit için 5 bitlik kod kelimesi

Girdi ikişer bit alınır; 2 bitle $2^2 = 4$ farklı durum vardır: `00`, `01`, `10`, `11`. Her birine 5 bitlik bir kod kelimesi verilir (ekrandan, 00:50:00):

Veri	Kod kelimesi
`00`	`00001`
`01`	`01010`
`10`	`10100`
`11`	`11111`

Kod kelimeleri, birinden diğerine geçerken en az 3 bit değişecek şekilde seçilmiştir. Hoca bunu sayarak gösterdi: birinciden ikinciye geçişte 3 bit, bir başka çiftte 4 bit değişir; hiçbir çift 3 bitten az farklı değildir. Amaç, bir bozulma olduğunda onu düzeltebilmektir.

Slayttaki tablo, her kod kelimesinin bütün tek bitlik bozulmuş hâllerini listeler:

Bozulmamış	`00001`	`01010`	`10100`	`11111`
1. bit bozuk	`10001`	`11010`	`00100`	`01111`
2. bit bozuk	`01001`	`00010`	`11100`	`10111`
3. bit bozuk	`00101`	`01110`	`10000`	`11011`
4. bit bozuk	`00011`	`01000`	`10110`	`11101`
5. bit bozuk	`00000`	`01011`	`10101`	`11110`

Bu 20 bozulmuş kelimenin hiçbiri başka bir sütunda ya da bozulmamış kelimeler arasında yer almaz. Bu yüzden bir bitte oluşan bozulmalar kesinlikle düzeltilebilir.

Örnek (ekrandan, 00:50:00). Orijinal mesaj `10 11 00 11` olsun. (Slaytta mesajın içinde bir de `01` vardı; hoca onun kodunu yazmayı atladığını söyleyip örnekten çıkardı.) Gönderilen dizi:

`10100 11111 00001 11111`

Üçüncü kelimenin birinci biti bozulsun; alınan dizi `10100 11111 10001 11111` olur. Alıcı ilk kelimeyi `10` olarak, ikinciyi `11` olarak açar. Üçüncü kelime `10001` diye bir kod yoktur. Alıcı tabloda arar: `10001` yalnız `00001`'in bozulmuş hâlleri arasında vardır. Kelimeyi `00001` ile değiştirir ve `00` olarak açar. Dördüncü kelime `11`'dir.

İki bit bozulsaydı, örneğin `11001` alınsaydı, bu kelime bozulmuş hâllerin hiçbirinde yoktur; orijinali bulunamaz, düzeltilemez.

> **Ek bilgi (derste anlatılmadı):** İki kod kelimesinin farklı olduğu bit sayısına Hamming uzaklığı denir. Bir koddaki en küçük uzaklık d_{min} ise kod $d_{min}-1$ bitlik hatayı sezebilir ve $\lfloor (d_{min}-1)/2 \rfloor$ bitlik hatayı düzeltebilir. Bu dersteki $2 \to 5$ kodunda ve Hamming (7,4) kodunda $d_{min} = 3$'tür; bu yüzden ikisi de tek bitlik hatayı düzeltir.

4.5 Ek bitlerin bedeli

Hoca sonra oranları karşılaştırdı: 1 biti 2 bitle gönderince hatayı yalnız buluyoruz, düzeltemiyoruz. 1 biti 3 bitle gönderince bir bitlik bozulmaları düzeltebiliyoruz. 2 biti 5 bitle gönderince de bir bitlik bozulmayı karşı taraftan istemeden çözebiliyoruz, ama fazladan bilgi ekliyoruz.

> **Ders sonrası not:** Hoca $2 \to 5$ kodu için $2/5$ oranını "%40 fazladan bilgi ekledim" diye, $1 \to 2$ için bir yerde "%100", bir yerde "%50 fazladan bilgi" diye andı. $2/5 = \%40$, kod kelimesinde veri bitlerinin payıdır. Eklenen 3 bit veriye göre $3/2 = \%150$, kod kelimesine göre $3/5 = \%60$'tır. $1 \to 2$'de veri payı $1/2 = \%50$, eklenen bit veriye göre $\%100$'dür; $1 \to 3$'te veri payı $1/3$'tür. Hamming (7,4) kodunda veri payı $4/7 \approx \%57$'dir; yani $2 \to 5$'ten daha az ek bitle aynı düzeltme gücüne ulaşılır.

Soru: 2 biti 5 bitle kodladık; 3 bit olsaydı kaç bitle kodlayacaktık? Bunun bir formülü var mı?

Cevap: Böyle bir formülasyon yoktur; kod kelimelerini çıkarmak tasarlayana kalmıştır. Önemli olan, her kod kelimesinin tek bitlik bozulmuş hâllerinin diğerlerininkiyle çakışmamasıdır; 3 bit için bunları oturup hesaplamak gerekir. Hoca literatürde böyle bir genellemenin yapılmadığını, daha iyi yöntemler olduğu için bunun üzerinde durulmadığını söyledi. Geliştirmeler sonunda Hamming kodlamaya varılmıştır: 4 bite 7 bitlik kod verir ve çok daha iyidir.

5. Hamming Kodlama

5.1 Tanım

Hamming kodlama, 4 bitlik verideki yer değiştirmelerle 7 bitlik kod kelimelerinin elde edildiği doğrusal (lineer) bir kodlamadır (ekrandan, 00:54:50). 1 bit yerine ortalama $7/4 = 1{,}75$ bit kullanılır. Slayttaki iki özellik:

- Bütün kod kelimelerinde en az 3 bitte değişim vardır.
- Bir bitte oluşan bozulmalar düzeltilebilir.

2 → 5 kodunda kod kelimeleri "en az 3 bit değişsin" diye elle seçiliyordu; Hamming kodunda bitler bir kurala göre hesaplanır.

5.2 Kodlama

7 bitlik kod kelimesinin ilk 4 biti orijinal veridir ($b_1 b_2 b_3 b_4$). Arkasına eklenen üç bit kafadan uydurulmaz; eşlik toplamıyla (parity addition) hesaplanır:

$$b_5 = b_1 \oplus b_2 \oplus b_3, \qquad b_6 = b_1 \oplus b_3 \oplus b_4, \qquad b_7 = b_2 \oplus b_3 \oplus b_4$$

$\oplus$ işlemi XOR gibi çalışır: girişler farklıysa (`01`, `10`) çıkış 1, aynıysa (`00`, `11`) çıkış 0'dır.

Örnek 1 (ekrandan, 00:54:50). Veri `1010`: $b_1 = 1, b_2 = 0, b_3 = 1, b_4 = 0$.

- $b_5 = 1 \oplus 0 \oplus 1$: $1 \oplus 0 = 1$, $1 \oplus 1 = 0$, yani $b_5 = 0$.
- $b_6 = 1 \oplus 1 \oplus 0$: $1 \oplus 1 = 0$, $0 \oplus 0 = 0$, yani $b_6 = 0$.
- $b_7 = 0 \oplus 1 \oplus 0$: $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, yani $b_7 = 1$.
- Kod kelimesi: `1010001`.

Örnek 2. Veri `0111`: $b_1 = 0, b_2 = 1, b_3 = 1, b_4 = 1$.

- $b_5 = 0 \oplus 1 \oplus 1 = 0$, $b_6 = 0 \oplus 1 \oplus 1 = 0$, $b_7 = 1 \oplus 1 \oplus 1 = 1$.
- Kod kelimesi: `0111001`.

Slayttaki tablonun tamamı (python ile doğrulandı):

Mesaj	Kod kelimesi	Mesaj	Kod kelimesi
0000	0000000	0110	0110010
0001	0001011	0101	0101110
0010	0010111	0011	0011100
0100	0100101	1110	1110100
1000	1000110	1101	1101000
1100	1100011	1011	1011010
1010	1010001	0111	0111001
1001	1001101		

Tabloda 15 mesaj vardır; 1111 slaytta yer almıyor (kuralla 1111111 olur).

5.3 Alıcıda eşlik kontrolü

Bozulmuş olabilecek kod kelimesi üzerine tilda konarak gösterilir: $\tilde{b}_1 \tilde{b}_2 \ldots \tilde{b}_7$. Alıcı üç eşlik kontrolü yapar. Bozulma yoksa üçü de sıfır çıkmalıdır:

$$P_1 = \tilde{b}_1 \oplus \tilde{b}_2 \oplus \tilde{b}_3 \oplus \tilde{b}_5$$
$$P_2 = \tilde{b}_1 \oplus \tilde{b}_3 \oplus \tilde{b}_4 \oplus \tilde{b}_6$$
$$P_3 = \tilde{b}_2 \oplus \tilde{b}_3 \oplus \tilde{b}_4 \oplus \tilde{b}_7$$

> **Ders sonrası not:** Slaytta (ekrandan, 01:00:00) $P_3 = \tilde{b}_2 \oplus \tilde{b}_3 \oplus \tilde{b}_4 \oplus \tilde{b}_5$ yazıyordu. Hoca örneği çözerken bunun hatalı olduğunu fark etti ve son terimin $\tilde{b}_7$ olması gerektiğini söyleyip slaytı düzeltti (ekrandan, 01:06:40). Doğru biçim, b_7'nin kodlama formülüyle uyumludur ve yukarıda yazılmıştır.

Hesaplanan P_1, P_2, P_3 değerlerine göre hangi bitin bozulduğu bulunur. Slayttaki kurallar (ekrandan, 01:00:00):

- $P_1 = P_2 = P_3 = 0$ ise $\tilde{b}_1 \dots \tilde{b}_7$ doğru kod kelimesidir. Dördüncü bitten sonrası ($\tilde{b}_5\tilde{b}_6\tilde{b}_7$) atılır, orijinal veri elde edilir.
- P_j'lerden biri 1, diğer ikisi 0 ise bozulan bit $\tilde{b}_l$'nin hangi P_j'den geldiği bulunur: ilgili P_j'de olup diğerlerinde olmayan $\tilde{b}_l$ seçilir ve tersi alınır. Örnek: $P_2 = 1$, $P_1 = P_3 = 0$ ise yalnız P_2'de bulunan $\tilde{b}_6$'nın tersi alınır.
- P_j'lerden biri 0, diğer ikisi 1 ise diğer ikisinde bulunup P_j'de olmayan $\tilde{b}_l$ bulunur ve tersi alınır. Örnek: $P_3 = 0$, $P_1 = P_2 = 1$ ise P_1 ve P_2'de olup P_3'te olmayan $\tilde{b}_1$'in tersi alınır. Hoca burada adayları tek tek eledi: b_2, b_3, b_4 ve b_5 ya P_3'te vardır ya da P_2'de yoktur; b_6 P_1'de yoktur; geriye b_1 kalır.
- Üçü de 1 ise üçünü birden bozan bir bit aranır: üçünde de bulunan $\tilde{b}_3$'tür.

Doğru denklemlerle her bitin hangi kontrollerde bulunduğu şöyledir (kısa tekrar için):

Bozulan bit	P_1	P_2	P_3
b_1	1	1	0
b_2	1	0	1
b_3	1	1	1
b_4	0	1	1
b_5	1	0	0
b_6	0	1	0
b_7	0	0	1

Her bitin deseni farklı olduğu için tek bitlik her bozulma ayırt edilir.

5.4 Çözülmüş örnek

Hoca slayttaki `0101` verisini kullandı (ekrandan, 01:06:40).

- Kodlama: $b_5 = 0 \oplus 1 \oplus 0 = 1$, $b_6 = 0 \oplus 0 \oplus 1 = 1$, $b_7 = 1 \oplus 0 \oplus 1 = 0$. Kod kelimesi `0101110`.
- Gönderilirken birinci bit bozuluyor; alınan kelime `1101110` ($\tilde{b}_1 = 1, \tilde{b}_2 = 1, \tilde{b}_3 = 0, \tilde{b}_4 = 1, \tilde{b}_5 = 1, \tilde{b}_6 = 1, \tilde{b}_7 = 0$).
- $P_1 = \tilde{b}_1 \oplus \tilde{b}_2 \oplus \tilde{b}_3 \oplus \tilde{b}_5 = 1 \oplus 1 \oplus 0 \oplus 1$: $1 \oplus 1 = 0$, $0 \oplus 0 = 0$, $0 \oplus 1 = 1$. $P_1 = 1$.
- $P_2 = \tilde{b}_1 \oplus \tilde{b}_3 \oplus \tilde{b}_4 \oplus \tilde{b}_6 = 1 \oplus 0 \oplus 1 \oplus 1$: $1 \oplus 0 = 1$, $1 \oplus 1 = 0$, $0 \oplus 1 = 1$. $P_2 = 1$.
- $P_3 = \tilde{b}_2 \oplus \tilde{b}_3 \oplus \tilde{b}_4 \oplus \tilde{b}_7 = 1 \oplus 0 \oplus 1 \oplus 0$: $1 \oplus 0 = 1$, $1 \oplus 1 = 0$, $0 \oplus 0 = 0$. $P_3 = 0$.
- P_3 doğru, P_1 ve P_2 hatalı. P_1 ve P_2'de olup P_3'te olmayan bit b_1'dir. $\tilde{b}_1$ 1 iken 0 yapılır, `0101110` ve ardından veri `0101` elde edilir.

Hoca bu örneği çözerken önce yanlış denklemle P_1 ve P_3'ü hatalı bulup "b_5 hem P_1'de hem P_3'te" sonucuna vardı. Sonucun tutmadığını görünce slaytta iki hata olduğunu fark etti: P_3'teki b_5'in b_7 olması gerekiyordu, bir de bir bit yazılmamıştı. Düzelttikten sonra yukarıdaki doğru sonuca ulaştı.

6. Eşlik Sınaması (Parity)

Aradan sonra hoca ARQ grubundaki yöntemlere geçti. Elektronik kökenliler ya da lisansta veri iletişimi dersi alanlar bu yöntemi daha önce duymuş olabilir.

Slayttaki tanımlar (ekrandan, 01:40:00):

- Gönderilen veride oluşan tek sayıdaki hatayı sezmek için kullanılır. "Tek sayıdaki" bir bitlik hata demek değildir: 1, 3, 5, 7 bitlik hatalar sezilir; çift sayıda bit bozulursa yapılacak bir şey yoktur.
- Verideki birlerin sayısını tek ya da çift olarak düzenler.
- Eşlik biti 1 ya da 0 yapılarak bütün veri grubundaki birlerin sayısının çift ya da tek olması sağlanır.

- **Tek eşlik** yönteminde veri içindeki 1'lerin sayısı tek ise eşlik biti 0 olur.
- **Çift eşlik** yönteminde veri içindeki 1'lerin sayısı tek ise eşlik biti 1 olur.

Algoritma hangi yöntemi kullandığını belirtir: "ben tek eşlik kullandım" ya da "çift eşlik kullandım".

Örnek (ekrandan, 01:40:00). Veri `0100010100`. İçinde 3 tane 1 vardır, yani tek sayıda.

- Tek eşlikte eşlik biti 0 olmalıdır; gönderilen `0100010100 0`.
- Çift eşlikte eşlik biti 1 olmalıdır; gönderilen `0100010100 1`.

Bozulma örneği. Tek eşlikle `0100010100 0` gönderilsin ve ikinci bit 1 iken 0 gelsin. Alıcı birlerin sayısını sayar: 2, yani çift. Tek eşlikte birlerin sayısı tek ise eşlik biti 0 olmalıydı; burada sayı çift olduğu hâlde eşlik biti 0'dır, demek ki bir bitte bozulma vardır. Ama hangi bit olduğu bilinmediği için düzeltilemez.

İki bit bozulsaydı (örneğin bir 1 daha 0 olsaydı ya da bir 0 1 olsaydı) birlerin sayısı yeniden tek olur, eşlik biti 0 ile uyumlu görünür ve bozulma bulunamaz. Üç bit bozulunca sayı yine tek-çift değiştirir ve sezilir. Yani yalnız tek sayıdaki bozulmalar sezilir. Hata sezilir ama düzeltilmez; bu yöntemler zaten hatanın karşı taraftan düzeltilmesini (verinin yeniden gönderilmesini) ister.

7. CRC (Cyclic Redundancy Check)

7.1 Üreteç fonksiyonu

CRC'de gönderilecek mesajın yanında bir üreteç fonksiyonuna, $G(x)$'e ihtiyaç vardır. $G(x)$ bir polinomdur; üçüncü, dördüncü, beşinci dereceden olabilir. Derecesi verinin boyutuna göre ayarlanır: veri çok büyükse ikinci ya da üçüncü dereceden üreteçler yetmeyebilir. Ayrıntıları literatürden okunabilir.

Slayttaki örnek (ekrandan, 01:46:50):

$$G(x) = x^3 + x^2 + 1 \quad \Rightarrow \quad G = 1101$$

`1101`, polinomun katsayılarıdır: x^3'ün katsayısı 1, x^2'nin 1, x^1 yok yani 0, x^0'ın 1. Üretecin derecesi 3 olduğu için orijinal veriye 3 bitlik CRC kodu eklenir; başlangıçta bu bitler `000`'dır. Slayttaki iki madde:

- $G(x)$: üreteç fonksiyonumuz.
- Üreteç fonksiyonunun derecesi CRC kodunun bit uzunluğunu verir ve başlangıçta değeri 0'dır.

7.2 Kodlama

Gönderilecek mesaj `0111101`; arkasına üç sıfır eklenir: `0111101 000`. Hocanın anlattığı ve slayttaki el yazısında görülen işlem (ekrandan, 01:46:50):

- Mesajın ilk 4 biti üreteçle XOR'lanır.
- XOR sonrasında en soldaki bit iptal edilir (atılır), bir sonraki bit aşağı indirilir.
- Yeni grubun en solundaki bite bakılır: 0 ise üreteç yerine `0000`, 1 ise üretecin katsayıları (`1101`) yazılır ve yeniden XOR yapılır.
- Bu, bütün bitler indirilene kadar sürer; sonda kalan 3 bit CRC kodudur.

Slayttaki satırlar sırasıyla şöyledir: `0111` ile `1101` XOR'lanır, `1010` çıkar; soldaki bit atılır. Sıradaki grup 0 ile başladığı için `0000` yazılır; sonra grup 1 ile başladığı için `1101` yazılır; böyle devam ederek sonunda `111` kalır. CRC kodu `111`'dir ve gönderilen mesaj:

`0111101 111`

> **Hocanın vurgusu:** Bazı kaynaklar CRC'yi bu şekilde anlatmaz. Burada önemli olan, XOR işleminden sonra en soldaki biti atıp sıradaki bite bakmaktır: 0 ise CRC kodu uzunluğu kadar sıfır, 1 ise orijinal üreteç yazılır.

> **Ders sonrası not:** Slayttaki işlemde ilk adımda baştaki bit 0 olduğu hâlde `0111` grubu `1101` ile XOR'lanmıştır. Standart CRC'de (modulo-2 polinom bölmesi) baştaki bit 0 iken `0000` ile XOR yapılır; bu kuralla `0111101000` mod `1101` kalanı `011` çıkar (python ile hesaplandı) ve gönderilen mesaj `0111101011` olur. Slayttaki `111` sonucu bu ilk adımdan kaynaklanır. Gönderen ve alıcı aynı işlemi uyguladığı için slayttaki örnekte de bozulma sezilir; ama standart bir CRC uygulamasıyla karşılaştırılacaksa `011` beklenmelidir.

7.3 Alıcıda kontrol

Alıcı gelen mesaja aynı işlemi uygular. Sonuçta orijinal CRC başlangıç değerine (sıfıra) dönülüyorsa bozulma yoktur.

Bozulmasız örnek (ekrandan, 01:50:00). Encode edilmiş mesaj `0111101111`. Hatalı mı gitti, doğru mu? Aynı adımlar uygulanır: `0111` ile `1101` XOR'lanır, soldaki bit atılır, sıradaki bite göre `0000` ya da `1101` yazılır. Slayttaki satırlar sonunda `0000` kalır: bozulma yoktur.

Bozulmuş örnek (ekrandan, 01:51:00). Aynı mesajın üçüncü biti 1 yerine 0 gelsin: `0101101111`. Üçüncü bit hatalı alınmış olsaydı anlaşılacak mıydı? Alıcı aynı işlemi yapar; bu kez sonda `0001` kalır. Bozulma olmasaydı sıfır çıkacaktı; sıfırdan farklı bir değer çıktığına göre bozulma vardır. Alıcı karşı tarafa "gönderdiğin veride bozulma var, yeniden gönder" der.

> **Ek bilgi (derste anlatılmadı):** CRC'nin matematiksel temeli şudur: mesaj bir polinom $M(x)$ olarak düşünülür, $x^r M(x)$ (sonuna r sıfır eklenmiş hâli) $G(x)$'e modulo-2 aritmetiğinde bölünür ve kalan $R(x)$ CRC olarak eklenir. Gönderilen $x^r M(x) + R(x)$ ifadesi $G(x)$'e tam bölünür; alıcıda kalanın sıfırdan farklı çıkması bozulmayı gösterir. Modulo-2 aritmetiğinde toplama ve çıkarma XOR'dur.

7.4 Üretecin seçimi

Soru: Bu üreteç fonksiyonu da sezgisel mi, kendimiz mi belirliyoruz? Bir formülü var mı?

Cevap: Formülasyonu yoktur. Kaynaklar yalnız üretecin derecesinin verinin büyüklüğüne göre ayarlanması gerektiğini söyler; hocanın kullandığı kitap ve yayınlarda daha fazla ayrıntı yoktur. Küçük veride 3. ya da 4. dereceden üreteç yeter; çok daha uzun bir bloğa CRC konacaksa daha yüksek dereceli bir üreteç seçilir. Bir iki denemeyle dosya büyüklüğüne göre en uygunu seçilir; sonra hep aynı üreteç kullanılır. Karşı tarafın da bu üreteci bilmesi gerekir, yoksa kontrol yapamaz. Üreteç sürekli değiştirilirse her değişikliğin karşı tarafa iletilmesi gerekir ve bu bir maliyettir; bu yüzden iki taraf belirli bir üreteçte anlaşır.

Hoca aynı durumun sıkıştırma algoritması seçiminde de geçerli olduğunu ekledi: veri tanındıkça hangi algoritmanın daha iyi sonuç verdiği anlaşılır; birkaç deneme yapılır ve sonunda birinde karar kılınır.

8. MD5 ve SHA

8.1 MD5'in kullanımı

MD5 (Message Digest) günlük hayatta çok kullanılan bir algoritmadır. Hoca bunu sözlü anlattı, çünkü güvenlikte kullanılan bir algoritmanın özet (hash) fonksiyonunun nasıl türetildiği açıkça anlatılmaz. Slayttaki maddeler (ekrandan, 02:00:00):

- Bir internet standardıdır, veri güvenliğinde ve bütünlüğünde kullanılır.
- Büyük bir veriden küçük bir özet oluşturulur.
- Tam güvenli bir algoritma değildir, ama hızlı çalıştığı için tercih edilir.
- İstenen her boyuttaki veriden 128 bitlik bir özet elde eder ve bu özeti 32 bitlik 4 eşit parçaya böler.
- MD5'in sıkıştırma fonksiyonunda bir çakışma bulunmasından yola çıkılarak SHA (Secure Hashing Algorithm) gibi algoritmalara yönelinmiştir.
- SHA-1, $(2^{64} - 1)$ bitlik mesajdan 160 bitlik özetleme değeri üretir.
- MD5 tek yönlü bir algoritmadır, geri dönüşümü yoktur.
- Veri 512 bitlik bloklara ayrılır ve her birine aynı işlem uygulanır; veri 512'nin katı değilse dolgu (padding) uygulanır.

Hocanın örneği şifrelerdir. Bir kuruma kullanıcı adı ve şifreyle girilir. Şifre unutulunca bazı sistemler şifreyi e-postayla gönderir: bu, şifreyi veritabanında açık olarak tutan güvensiz bir sistemdir. Alışveriş siteleri gibi iyi sistemler şifreyi göndermez, sıfırlamayı ister. Çünkü şifre MD5 gibi bir algoritmayla özetlenerek saklanır; kullanıcı şifresini girdiğinde girilen şifre aynı algoritmadan geçirilir ve veritabanındaki özetle karşılaştırılır. Aynıysa doğru kişidir. Özeti açacak bir bilgi olmadığı için sistem şifreyi geri veremez, yenisini oluşturmayı ister.

Hoca özet fonksiyonunu veri yapıları dersindeki hash fonksiyonlarına benzetti: bir değeri dizide saklarken önce bir hash fonksiyonundan geçirmek gibi, burada da string bir özet fonksiyonundan geçirilir.

8.2 Çakışma ve SHA

MD5 bazen farklı girdiler için aynı özeti üretebilir; bu çok nadirdir (hoca "milyarda ya da milyonda bir" dedi). İnternet standartlarında bu göz ardı edilir, çünkü algoritma çok hızlıdır. Çakışmalar nedeniyle MD5'in tam güvenli olmadığı söylenmiş ve SHA önerilmiştir. SHA da bir özet fonksiyonudur; en çok $2^{64} - 1$ bitlik bir mesajdan 160 bitlik özet üretir. MD5 ise girdinin boyutu ne olursa olsun 128 bitlik özet üretir. Hoca, SHA daha yavaş çalıştığı için MD5'in hâlâ kullanıldığını söyledi. Dolgu işlemini de "sayı oluşuncaya kadar arkasına 0 biti eklemek" diye anlattı.

> **Ders sonrası not:** Anlatımda ve slaytta bir yerde "özet bilgiden orijinal veriye dönülür" ifadesi geçti; aynı slaytta MD5'in tek yönlü olduğu, geri dönüşümü olmadığı da yazıyor. Doğrusu ikincisidir: özetten orijinal veri elde edilmez, şifre kontrolü özetlerin karşılaştırılmasıyla yapılır. Hoca da şifre örneğinde sistemin şifreyi geri veremediğini anlattı.
>
> **Ders sonrası not:** Hoca MD5'in kırılabildiğinin ispat edilmediğini ve MD5'in SHA'dan "daha kuvvetli" olduğunu söyledi. MD5 için pratik çakışma üretme yöntemleri büyük olasılıkla 2004'ten beri bilinmektedir; MD5 bugün şifre saklama ve imza gibi güvenlik amaçları için önerilmez, SHA-2 ailesi gibi daha güçlü özet fonksiyonları kullanılır. Hocanın "her 128 bitlik veriye bir özet, 128'in katlarına ayırır" dediği yerde slayttaki doğru bilgi 512 bitlik bloklar ve 128 bitlik özettir. MD5'in dolgusu da yalnız sıfırlardan oluşmaz; önce bir 1 biti, sonra sıfırlar ve en sonda mesajın uzunluğu eklenir.

9. Günlük Hayattan Kontrol Rakamlı Kodlar: UPC ve POSTNET

Hoca, kitaplarda sayısız kodlama yöntemi bulunduğunu, kendisinin günlük hayatta karşılaşılanları seçtiğini söyledi.

9.1 UPC (Universal Product Code)

Ürünlerin arkasındaki bar kodlardır. İnce, biraz daha kalın, kalın ve çok kalın siyah çizgiler ile aynı şekilde farklı genişlikte boşluklar vardır; göz bunları kolayca ayırt edemese de her birinin kodu önceden belirlenmiştir. Bar kodun üzerinde hem çizgiler hem rakamlar bulunur. Her ondalık rakam 7 ikili bitle (7 bit bir araya gelip bir rakamı oluşturur) kodlanır. Ortada bir ayırıcı vardır; solundaki ve sağındaki rakamlar için ayrı kod tabloları kullanılır (ekrandan, 02:03:30):

Rakam	Sol	Sağ
0	`0001101`	`1110010`
1	`0011001`	`1100110`
2	`0010011`	`1101100`
3	`0111101`	`1000010`
4	`0100011`	`1011100`
5	`0110001`	`1001110`
6	`0101111`	`1010000`
7	`0111011`	`1000100`
8	`0110111`	`1001000`

Rakam	Sol	Sağ
9	`0001011`	`1110100`

Slayttaki notlar:

- Sol taraftaki her 7'li grup 0 ile başlar, 1 ile biter. Sağ taraftaki her 7'li grup 1 ile başlar, 0 ile biter.
- Sol taraftaki her 7'li grupta birlerin sayısı tektir; sağ taraftakilerde çifttir.

Slayttaki ürünün bar kodu `04850000139 4` olsun. Sayı UPC bar code ile 6 + 6 rakam olarak kodlanmıştır; rakamların dağılımı 1 + 5 + 5 + 1'dir:

- 1 rakam: ürünün tipi,
- 5 rakam: üreticinin kodu,
- 5 rakam: ürünün kodu,
- 1 rakam: kontrol rakamı. Kontrol rakamı bar kodun içinde yer alır ama genelde altına ondalık sayı olarak yazılmaz.

Hoca anlatırken bunlara "bit" dedi, ama kastettiği rakamlardır. Bar kod okunurken kontrol rakamıyla okumanın hatalı mı hatasız mı olduğu kontrol edilir.

Kontrol rakamının hesabı (ekrandan, 02:07:40). İlk 11 rakam alınır:

$$3a_1 + a_2 + 3a_3 + a_4 + 3a_5 + a_6 + 3a_7 + a_8 + 3a_9 + a_{10} + 3a_{11} \equiv A \pmod{10}$$

Kontrol rakamı C (ya da a_{12}) ise $C = 10 - A$'dır.

`04850000139` için: $a_1 = 0, a_2 = 4, a_3 = 8, a_4 = 5, a_5 = 0, a_6 = 0, a_7 = 0, a_8 = 0, a_9 = 1, a_{10} = 3, a_{11} = 9$.

- $3 \cdot 0 = 0$, $a_2 = 4$, $3 \cdot 8 = 24$, $a_4 = 5$, $3 \cdot 0 = 0$, $a_6 = 0$, $3 \cdot 0 = 0$, $a_8 = 0$, $3 \cdot 1 = 3$, $a_{10} = 3$, $3 \cdot 9 = 27$.
- Toplam: $4 + 24 + 5 + 3 + 3 + 27 = 66$.
- $66 \bmod 10 = 6$, yani $A = 6$.
- $C = 10 - 6 = 4$.

Kontrol rakamı 4'tür; bar kodda yazılmasa da içinde 4 yer alır. Çizgi ve boşluklar hatalı okunursa rakamlar farklı algılanır, çarpımlar farklı gelir ve kontrol rakamı farklı çıkar; böylece hatalı okuma anlaşılır.

Hoca "kontrol rakamı da bozulmaz mı?" sorusunu kendisi sordu ve cevapladı: bozulabilir. Ama onu kontrol etmek için bir kontrol rakamı daha, onu kontrol etmek için bir tane daha konamaz; kontrol rakamıyla orijinal veride bozulma olup olmadığı kontrol edilir.

9.2 POSTNET (ABD posta bar kodu)

Amerika'da posta sisteminde kullanılan başka bir kodlamadır (ekrandan, 02:10:00). 0-9 arasındaki ondalık rakamları kodlar; her rakam için 5 çizgi ayrılır. Çizgiler kısa ve uzundur: kısa yerine 0, uzun yerine 1 konur. Her kodda 3 kısa ve 2 uzun çizgi vardır. Baştaki ve sondaki uzun çizgiler yalnız sınır (guard) çizgileridir.

Rakam	Kod	Rakam	Kod
1	`00011`	6	`01100`
2	`00101`	7	`10001`
3	`00110`	8	`10010`
4	`01001`	9	`10100`
5	`01010`	0	`11000`

Slayttaki zarfın bar kodundan okunan sayı `2007755761`'dir; bu, ZIP+4 kodu `20077-5576`'ya karşılık gelir ve son rakam 1 kontrol rakamıdır. Bu algoritmada kontrol rakamı farklı biçimde kullanılır: kontrol rakamı dahil bütün rakamların toplamı 10'a bölünebilmelidir.

$$a_1 + a_2 + \cdots + a_9 + a_{10} \equiv 0 \pmod{10}$$

Okunamayan rakam örneği. Hoca ikinci 7'nin okunamadığını varsaydı ve yerine soru işareti koydu: `2 0 0 7 ? 5 5 7 6 1`. Bilinen rakamların toplamı:

$$2 + 0 + 0 + 7 + 5 + 5 + 7 + 6 + 1 = 33$$

Toplamın 10'a tam bölünmesi için $33 + ? = 40$ olmalıdır, yani $? = 7$. Okunamayan rakam bu şekilde bulunur.

Çizgilerden biri bozulursa, örneğin bir kısa çizgi uzun okunursa `100 11` gibi tabloda olmayan bir kod çıkar; bunun yerine hiçbir rakam konamaz ve okunamayan rakam yukarıdaki gibi tahmin edilir.

Hoca, farklı uygulamalarda kullanılan onlarca kod bulunabileceğini, her birinin kontrol bitinin nasıl hesaplandığını, kodların nasıl türetildiğini ve ters işlemlerin nasıl yapıldığını gösteren kendi algoritması olduğunu söyledi; bu dersin içeriği için bunlara gerek yoktur.

10. Sezgisel Sıkıştırma: Braille, Mors, Kısaltmalar

Hata konusunu toparladıktan sonra hoca üçüncü bölüme, temel tekniklere geçti. Terminoloji, veri güvenliği ile sıkıştırma arasındaki ikilem, bozulmaların nasıl sezildiği ve bir bit için kaç bit verilmesi gerektiği görülmüştü. Ortada "veri sıkıştırma" diye bir kavram yokken bile insanlar bir şeyleri kodlamak istemiştir; bu kodlamalar aslında sıkıştırma amaçlıdır ve kodu bilmeyen için anlamsızdır. Slayttaki başlık: Sezgisel Sıkıştırma (Intuitive Compression) (ekrandan, 02:20:40).

10.1 Braille kodlaması

Braille, görme engellilerin kullandığı alfabedir; "6 nokta" diye de geçer. 6 noktayla $2^6 = 64$ farklı sembol yazılabilir. A'dan Z'ye 26 harf, 0-9 arası 10 rakam ve nokta, virgül, soru işareti gibi yaklaşık 10 işaret eklense bile 46 civarında sembol eder; hâlâ boş desenler kalır. Bu desenler dilde çok kullanılan kelimelere ya da harf gruplarına verilebilir. Slayttaki İngilizce örnekte "and", "for", "of", "the", "with" kelimeleri ve "ch", "gh", "sh", "th" ikilileri tek hücreyle kodlanır. Böylece iki harf ya da bir kelime 6 bitle ifade edilir.

Noktalar 1'den 6'ya numaralanırsa "A"nın kodu yalnız 1. noktadır: `100000`. Hoca küçük/büyük harf ayrımına neden girilmediğini de açıkladı: kodları boşuna harcamamak için. Bunun yerine kullanılmayan bir sembol "büyük harf işareti" olarak seçilir; o işaret görüldüğünde arkasından gelen karakterler büyük harf okunur. Benzer şekilde bir küçük harf işareti de konabilir.

Braille kabartılı yazıdır. Bilgisayara aktarılmak istenirse doküman görüntü olarak alınır; kabartılar resimde koyu ya da gri çıkar. Görüntü analiz edilip kabartı olan yerlere 1, olmayan yerlere 0 verilir ve veri işlenebilir yazıya dönüştürülür.

Braille'in farklı türleri vardır (ekrandan, 02:20:40):

- Literary: günlük hayatta kullanılan türde yazılar için.
- Nemeth: matematik ve fen bilimleri dokümanları için; delta, beta, gamma gibi semboller içerir.
- CBC (Computer Braille Code): görme engellilerin program kodu yazarken kullandığı Braille kodu.
- Music: notaları okumak için geliştirilmiş Braille kodu; müzikteki semboller ve notaların karşılıkları vardır.

10.2 Mors kodu

Mors kodu, 1838 yılında Samuel Morse tarafından geliştirilmiş bir telgraf kodudur. Hoca eski Amerikan filmlerindeki "tık tık" diye çekilen telgrafları hatırlattı. İlk başta uzun ve kısa çizgilerden oluşuyordu ve kodlar biraz karışıktı. Bir iki sene sonra Alfred Vail (altyazıda "Alfred Fail")

daha okunaklı ve kullanışlı olması için yalnız çizgi ve noktalardan oluşan yeni Mors alfabesini ortaya attı.

Bir noktanın ya da çizginin hangi karaktere ait olduğu, bir kelimenin ne zaman bittiği zamanlamayla anlaşılır. Slayttaki kurallar (ekrandan, 02:20:40):

- Her nokta 1 birim, her çizgi 3 birimdir.
- Nokta ve çizgiler arası 1 birimdir.
- Karakterler arası 3, kelimeler arası 6 birimdir.
- Hata yapıldığında son kelimenin silinmesi için 8 nokta gönderilir.

Mors alfabesi kulakla dinlenerek anlaşılabildiği gibi ses işlenerek Latin harflerine de çevrilebilir. Yalnız harflerin değil, rakamların ve bazı noktalama işaretlerinin de kodları vardır.

> **Ders sonrası not:** Uluslararası Mors kodunda kelimeler arası boşluk büyük olasılıkla 7 birimdir; slayttaki 6 birim bu standarttan farklıdır.

10.3 Kısaltmalar ve akronimler

Günlük hayatta kullanılan akronimler de sıkıştırmadır. “Yıldız Teknik Üniversitesi” yerine “YTÜ” yazılır. Hoca karakterleri kabaca saydı (Yıldız 6, Teknik 6, Üniversite yaklaşık 10, toplam yaklaşık 22) ve bunların 3 karaktere indirildiğini söyledi; farkında olmadan bir sıkıştırma yapılmış olur. Akronim ile kısaltma farklıdır: akronimde her harf bir kelimeye karşılık gelir; “Doktor” için “Dr.” yazmak kısaltmadır, akronim değildir.

> **Ders sonrası not:** “Yıldız Teknik Üniversitesi” boşluksuz 24 harftir (6 + 6 + 12); hoca “Üniversite”yi yaklaşık 10 alarak 22 dedi.

Hoca, bu “tık” kodlamalarının da bir sıkıştırma olduğunu vurguladı: ASCII’de “A” 8 bitle ifade edilirken Braille’de 6 bitle ifade edilir. Mors’ta da bir karakter, çizgiler 3 birim sayılınca, birkaç birimle ifade edilir.

11. Sparse String Sıkıştırma

Hocanın ilgisini çeken bir yöntemdir; anlatmayı kolaylaştırmak için slaytı renkli hazırlamıştır. Sparse string, boşluklarla dolu, boşlukların çok fazla olduğu metindir. Slayttaki örnek metin (ekrandan, 02:25:50):

```
Bu__dersin__adı___veri_sıkıştırma___ve_pazatesi_günü__yapılıyormuş.
```

Burada _ boşluğu gösterir. Boşluklar silinip geçilirse orijinal metne dönülemez. Algoritma şöyle çalışır:

- Metinde karakter geçen yere 0, boşluk geçen yere 1 verilir.
- Bitler 4’lü gruplara ayrılır; bu birinci seviyedir (L1).
- L2, L1’in her grubu için bir bittir: grup sıfırdan farklıysa 1, `0000` ise 0.
- L3 de L2’nin grupları için aynı şekilde kurulur.
- L1 ve L2’deki sıfır olan gruplar silinir; L1, L2, L3 dizileri ve ardından boşluksuz metin gönderilir.

Slayttaki değerler:

```
L1 = 0011|0000|0011|0001|0001|0000|0000|0011|1001|0000|0000|1000|0110|0000|0000|0000
L2 = 1011|1001|1001|1000
L3 = 1111

Sıfır gruplar silindikten sonra:
L1 = 0011|0011|0001|0001|0011|1001|1000|0110
L2 = 1011|1001|1001|1000
L3 = 1111
Gönderilen: L1 L2 L3 budersiadıverisıkıştırmavepazartesigünüyapılıyormuş
```

Slaytın altındaki ağaç, L3’ün her bitinin bir L2 grubunu, L2’nin her bitinin de bir L1 grubunu gösterdiğini anlatır: L3 = `1111`’in dört biti dört L2 grubuna (`1011`, `1001`, `1001`, `1000`), bunların

her biti de dörder L1 grubuna bağlanır. Açarken L3'ten başlanır; 0 olan bitlerin gösterdiği gruplar `0000` olarak geri konur.

Hoca L1'i harf harf kurarken ("Bu" iki karakter, iki boşluk: `0011`; "ders": `0000`; "in" ve iki boşluk: `0011`…) slaytta bir hata olduğunu fark etti. Düzeltmenin zaman alacağını söyleyip dersi burada bitirdi; konuya sonraki derste devam edilecek.

> **Ders sonrası not:** Slayttaki L1 dizisinin ilk dört grubu metinle uyumludur, ama sonraki grupların bir kısmı yukarıdaki metinle büyük olasılıkla uyuşmuyor (hoca da bir hata olduğunu söyledi). L2 ve L3, slayttaki L1'e göre doğru kurulmuştur. Doğru değerler sonraki derste verilecektir.

Örnek Sorular

Soru 1 (derste sözlü, 00:06:30 - 00:14:30): Önceki dersin tekrarı olarak sorulan sorular. Veri sıkıştırma nedir? Sabit ve değişken uzunluklu kod ne demektir? Bir yöntemin adaptif ya da non-adaptif olması nedir? Kayıplı ve kayıpsız algoritma ne demektir? Simetrik ve asimetrik algoritma nedir?

> **Çözüm (derste verildi):** Veri sıkıştırma, yerden ve zamandan kazanmak için yapılan işlemdir. Sabit uzunluklu kodda bütün sembollere aynı bit sayısı, değişken uzunluklu kodda sıklığa göre farklı uzunlukta kod verilir. Non-adaptif yöntem belli tipte veride çalışır; adaptif yöntem her tür veride çalışır, parametreleri değişebilir. Kayıplıda çözülen veri orijinale dönmez (görüntü, ses); kayıpsızda birebir döner (metin). Simetrikte açma, kodlama algoritmasının tersten çalıştırılmasıdır; asimetrikte iki taraf farklı algoritmadır ve hızları farklı olabilir. Ayrıntılar Bölüm 1'dedir.

Soru 2 (ekrandan, 00:40:00; derste sözlü 00:34:50): Gönderilen veri `10110100110`, alınan veri `10100100110`'dır. Alıcı bu veride bir bozulma olduğunu anlayabilir mi?

> **Çözüm (derste verildi):** Anlayamaz. Dördüncü bitin orijinalinin 1 olduğunu bilemez, 0 olarak alır ve açar. Ek bit olmadan hata sezilemez. Her bit iki kez gönderilirse hata sezilir ama düzeltilemez; üç kez gönderilirse tek bitlik hata düzeltilir.

Soru 3 (ekrandan, 00:50:00): Her 2 bit için `00` → `00001`, `01` → `01010`, `10` → `10100`, `11` → `11111` kodları kullanılıyor. `10 11 00 11` mesajı gönderiliyor ve alıcıya `10100 11111 10001 11111` geliyor. Alıcı mesajı nasıl çözer?

> **Çözüm (derste verildi):** `10100` → `10`, `11111` → `11`. `10001` diye bir kod yoktur; tek bitlik bozulmalar tablosunda bu kelime yalnız `00001`'in bozulmuş hâlleri arasında bulunur, alıcı onu `00001` ile değiştirip `00` olarak çözer. Son kelime `11`'dir. Mesaj: `10 11 00 11`. İki bit bozulsaydı (örneğin `11001`) tabloda karşılığı olmadığı için düzeltilemezdi.

Soru 4 (ekrandan, 01:06:40): Hamming (7,4) koduyla `0101` verisi kodlanıp gönderiliyor; alıcıya `1101110` geliyor. Bozulma var mı, varsa hangi bit?

> **Çözüm (derste verildi):** Kodlama `0101110`'dur. Alınan kelime için $P_1 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$, $P_2 = 1 \oplus 0 \oplus 1 \oplus 1 = 1$, $P_3 = 1 \oplus 0 \oplus 1 \oplus 0 = 0$. P_1 ve P_2'de olup P_3'te olmayan bit b_1'dir. b_1'in tersi alınır: `0101110`, veri `0101`.

Soru 5 (ekrandan, 01:50:00): $G(x) = x^3 + x^2 + 1$ üreteciyle encode edilmiş mesaj `0111101111` alınıyor. Hatalı mı gitti, doğru mu?

> **Çözüm (derste verildi):** Alıcı, kodlamadaki işlemi (XOR, soldaki biti atma, sıradaki bite göre `1101` ya da `0000` yazma) gelen mesaja uygular; sonda `0000` kalır. Bozulma yoktur.

Soru 6 (ekrandan, 01:51:00): Aynı mesajın üçüncü biti hatalı alınmış olsaydı (`0101101111`), bu anlaşılacak mıydı?

> **Çözüm (derste verildi):** Evet. Aynı işlem sonunda `0001` kalır. Bozulma olmasaydı sıfır çıkacaktı; sıfırdan farklı bir değer çıktığı için bozulma vardır ve alıcı verinin yeniden gönderilmesini ister.

Soru 7 (ekrandan, 02:07:40): UPC bar kodunun ilk 11 rakamı `04850000139` ise kontrol rakamı kaçtır?

Çözüm (derste verildi): $3\cdot 0+4+3\cdot 8+5+3\cdot 0+0+3\cdot 0+0+3\cdot 1+3+3\cdot 9=66$. $66 \bmod 10 = 6 = A$, kontrol rakamı $C = 10-6 = 4$.

Soru 8 (ekrandan, 02:10:00; derste sözlü 02:10:30): POSTNET bar kodundan `2 0 0 7 ? 5 5 7 6 1` okunuyor; beşinci rakam okunamadı (son rakam kontrol rakamı). Okunamayan rakam kaçtır?

Çözüm (derste verildi): Kontrol rakamı dahil bütün rakamların toplamı 10'a bölünebilmelidir. Bilinen rakamların toplamı 33'tür; $33 + ? = 40$ için $? = 7$.

Dersin Toparlaması

Bu derste önce önceki haftanın temel kavramları (sıkıştırmanın amacı, sabit/değişken kod, adaptif/non-adaptif, kayıplı/kayıpsız, simetrik/asimetrik, derlem, başarı ölçütleri) tekrar edildi. Sonra sıkıştırmadan kısa bir ara verilerek hata bulma ve düzeltme işlendi. Ana fikir, sıkıştırma ile güvenlik arasındaki ikilemdir: hatayı sezmek ve düzeltmek için veriye ek bit eklemek gerekir ve bu sıkıştırma kazancını azaltır. Hata türleri (burst, random, spike, erasure) ve iki yaklaşım görüldü: FEC'te alıcı hatayı kendisi düzeltir (Hamming), ARQ'da hatayı sezip verinin yeniden gönderilmesini ister (eşlik, CRC). Tekrarlama kodlarından 2 → 5 bit koduna ve Hamming (7,4) koduna geçilerek, kod kelimeleri arasındaki en az 3 bitlik farkın tek bitlik hatayı düzeltmeyi nasıl sağladığı gösterildi. MD5/SHA özet fonksiyonları ile UPC ve POSTNET bar kodlarındaki kontrol rakamları günlük hayattan örnekler olarak anlatıldı.

Ardından temel sıkıştırma tekniklerine giriş yapıldı: Braille, Mors kodu ve akronimler gibi sezgisel sıkıştırmalar. Son olarak sparse string sıkıştırmanın fikri tanıtıldı; slayttaki bir hata nedeniyle konu yarım kaldı. Hoca sonraki derste sparse string sıkıştırmadan devam edileceğini, bu bölümdeki üç yöntem görüldükten sonra bilgi teorisine (information theory) ve ardından RLE'ye (Run Length Encoding) geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.